O'REILLY®

Professional Cloud Architect: Day 1 Polls

O'REILLY®

Introduction and Resource Hierarchy

Introduction to Google Cloud

As a cloud engineer frequently moving between different computers and locations, you require consistent, daily access to your Google Cloud project's resources, specifically BigQuery, Bigtable, and Kubernetes Engine. Your priority is to have a ready-to-use command-line environment with the gcloud tool without the overhead of installing, configuring, and maintaining it on every machine you use. Which of the following provides the most efficient and hassle-free solution?

A. Install the gcloud CLI on each workstation you use and create a script to automate the update process across all machines.
B. Utilize a system package manager, such as apt or yum, to streamline the gcloud CLI installation process on each of your different workstations.
C. Leverage the browser-accessible, pre-configured Cloud Shell environment provided within the Google Cloud Console for all your tasks.
D. Utilize a system package manager, such as apt or yum, to streamline the gcloud CLI installation process on each of your different workstations.

Introduction to Google Cloud

As a cloud engineer frequently moving between different computers and locations, you require consistent, daily access to your Google Cloud project's resources, specifically BigQuery, Bigtable, and Kubernetes Engine. Your priority is to have a ready-to-use command-line environment with the gcloud tool without the overhead of installing, configuring, and maintaining it on every machine you use. Which of the following provides the most efficient and hassle-free solution?

A. Install the gcloud CLI on each workstation you use and create a script to automate the update process across all machines.
B. Utilize a system package manager, such as apt or yum, to streamline the gcloud CLI installation process on each of your different workstations.
C. Leverage the browser-accessible, pre-configured Cloud Shell environment provided within the Google Cloud Console for all your tasks.
D. Utilize a system package manager, such as apt or yum, to streamline the gcloud CLI installation process on each of your different workstations.

Introduction to Google Cloud

Your organization has several departments, each with development teams requiring similar access permissions within their department. Each department also maintains separate dev, test, and prod environments to isolate workloads. You need to design a Google Cloud resource hierarchy that ensures clear separation of environments, consistent permission management, and ease of administration across departments. Which approach best meets these requirements?

A. Create a single project for each department and manage dev, test, and prod environments by using different VPC networks or subnets within that project to isolate workloads.

B. Use folders to represent each department, and create separate projects under each folder for dev, test, and prod environments. Assign permissions at the folder and project levels for consistent and isolated access control.

C. Have all environments (dev, test, prod) for all departments inside a single project, using labels and resource tagging to distinguish between them and control permissions accordingly.

D. Design separate VPCs for each environment (dev, test, prod) across all departments, while keeping all projects centralized under one folder representing the entire organization.

Introduction to Google Cloud

Your organization has several departments, each with development teams requiring similar access permissions within their department. Each department also maintains separate dev, test, and prod environments to isolate workloads. You need to design a Google Cloud resource hierarchy that ensures clear separation of environments, consistent permission management, and ease of administration across departments. Which approach best meets these requirements?

A. Create a single project for each department and manage dev, test, and prod environments by using different VPC networks or subnets within that project to isolate workloads.

B. Use folders to represent each department, and create separate projects under each folder for dev, test, and prod environments. Assign permissions at the folder and project levels for consistent and isolated access control.

C. Have all environments (dev, test, prod) for all departments inside a single project, using labels and resource tagging to distinguish between them and control permissions accordingly.

D. Design separate VPCs for each environment (dev, test, prod) across all departments, while keeping all projects centralized under one folder representing the entire organization.

O'REILLY®

Compute Engine

Compute Instances

Your company runs batch processing workloads on Google Cloud Platform, some of which are not time-sensitive. Additionally, your organization must comply with strict financial data regulations requiring the use of certified GCP services. You also want to manage infrastructure costs effectively while ensuring compliance. How should you design your solution following Google best practices?

A. Use preemptible VMs to minimize costs and immediately stop using all GCP services that do not meet the financial compliance requirements.

B. Deploy non-preemptible standard VMs in appropriate regions to ensure workload stability and compliance, while carefully selecting only certified GCP services for processing sensitive data.

C. Consolidate all workloads into a single region using preemptible VMs and apply labels to track compliance, without limiting service usage based on certification status.

D. Use standard VMs with auto-scaling in multiple regions to reduce costs, but continue using all GCP services regardless of compliance certification to maintain flexibility.

Compute Instances

Your company runs batch processing workloads on Google Cloud Platform, some of which are not time-sensitive. Additionally, your organization must comply with strict financial data regulations requiring the use of certified GCP services. You also want to manage infrastructure costs effectively while ensuring compliance. How should you design your solution following Google best practices?

A. Use preemptible VMs to minimize costs and immediately stop using all GCP services that do not meet the financial compliance requirements.

B. Deploy non-preemptible standard VMs in appropriate regions to ensure workload stability and compliance, while carefully selecting only certified GCP services for processing sensitive data.

C. Consolidate all workloads into a single region using preemptible VMs and apply labels to track compliance, without limiting service usage based on certification status.

D. Use standard VMs with auto-scaling in multiple regions to reduce costs, but continue using all GCP services regardless of compliance certification to maintain flexibility.

Compute Instances

Your company plans to migrate several Linux virtual machines running on an on-premises VMware infrastructure to Google Cloud Compute Engine. You want to follow Google-recommended best practices for a reliable and efficient migration. What approach should you take?

A. 1. Inventory the applications and their dependencies on each VM. 2. Export the VMs as disk images, upload to Google Cloud Storage, and create Compute Engine instances using these images.

B. 1. Assess the existing VMware virtual machines and their workloads. 2. Prepare a detailed migration plan, create a migration runbook for Migrate for Compute Engine, and perform the migration using this service.

C. 1. Install third-party migration software agents on all Linux VMs. 2. Use these agents to replicate and migrate VMs manually to Compute Engine 3. Use Migrate for Compute Engine for verification.

D. 1. Perform a quick assessment and shut down all VMs at once. 2. Create snapshots and manually import them as boot disks using Migrate for Compute Engine

Compute Instances

Your company plans to migrate several Linux virtual machines running on an on-premises VMware infrastructure to Google Cloud Compute Engine. You want to follow Google-recommended best practices for a reliable and efficient migration. What approach should you take?

A. 1. Inventory the applications and their dependencies on each VM. 2. Export the VMs as disk images, upload to Google Cloud Storage, and create Compute Engine instances using these images.

B. 1. Assess the existing VMware virtual machines and their workloads. 2. Prepare a detailed migration plan, create a migration runbook for Migrate for Compute Engine, and perform the migration using this service.

C. 1. Install third-party migration software agents on all Linux VMs. 2. Use these agents to replicate and migrate VMs manually to Compute Engine 3. Use Migrate for Compute Engine for verification.

D. 1. Perform a quick assessment and shut down all VMs at once. 2. Create snapshots and manually import them as boot disks using Migrate for Compute Engine

Compute Instances

A media company runs a video processing application on Google Compute Engine. The application stores its working data on attached disks and must resume operation quickly in another zone if the current zone experiences an outage. The company wants to minimize downtime and avoid data loss during such events while keeping the recovery process automated and efficient. How should they design this setup?

A. Schedule frequent backups of the VM's boot and data disks to Cloud Storage, and after a zone failure, restore the backups manually and recreate the VM in the same zone.

B. Deploy the application using an instance template and attach a regional persistent disk to the VM. If the current zone goes down, automatically launch a new instance in another zone in the same region with the attached regional persistent disk.

C. Use zonal persistent disks for the application's data and replicate data asynchronously to a secondary zone. In case of failure, manually detach and attach the disk to a new VM in the other zone.

D. Set up the application on multiple VMs across zones using local SSDs and rely on application-level replication for data recovery in case of a zone outage.

Compute Instances

A media company runs a video processing application on Google Compute Engine. The application stores its working data on attached disks and must resume operation quickly in another zone if the current zone experiences an outage. The company wants to minimize downtime and avoid data loss during such events while keeping the recovery process automated and efficient. How should they design this setup?

A. Schedule frequent backups of the VM's boot and data disks to Cloud Storage, and after a zone failure, restore the backups manually and recreate the VM in the same zone.

B. Deploy the application using an instance template and attach a regional persistent disk to the VM. If the current zone goes down, automatically launch a new instance in another zone in the same region with the attached regional persistent disk.

C. Use zonal persistent disks for the application's data and replicate data asynchronously to a secondary zone. In case of failure, manually detach and attach the disk to a new VM in the other zone.

D. Set up the application on multiple VMs across zones using local SSDs and rely on application-level replication for data recovery in case of a zone outage.

Compute Instances

Your company is migrating a legacy financial application to Google Cloud. This application has a strict, per-physical-server licensing model. To maintain compliance, the Virtual Machines running this application must be confined to a specific, pre-paid set of physical hosts.

You have created a single sole-tenant node group for the finance department. Within this group, you have provisioned two specific nodes (`licensed-host-01` and `licensed-host-02`) that are designated solely for this licensed application, while other nodes in the same group are for general use.

When you create a new VM instance for this licensed application, how do you ensure it is scheduled *only* on one of the designated licensed hosts and not on any other nodes in the group?

A. When creating the VM, apply a network tag that matches the name of the sole-tenant node group.

B. When creating the VM, configure a custom metadata key named `gce-schedule-on-host` with the value `licensed-host-01`.

C. When creating the VM, define a node affinity label that requires the VM to be scheduled on any node within the finance department's node group.

D. When creating the VM, define a node affinity label that specifically targets the node names `licensed-host-01` or `licensed-host-02`.

Compute Instances

Your company is migrating a legacy financial application to Google Cloud. This application has a strict, per-physical-server licensing model. To maintain compliance, the Virtual Machines running this application must be confined to a specific, pre-paid set of physical hosts.

You have created a single sole-tenant node group for the finance department. Within this group, you have provisioned two specific nodes (`licensed-host-01` and `licensed-host-02`) that are designated solely for this licensed application, while other nodes in the same group are for general use.

When you create a new VM instance for this licensed application, how do you ensure it is scheduled *only* on one of the designated licensed hosts and not on any other nodes in the group?

A. When creating the VM, apply a network tag that matches the name of the sole-tenant node group.

B. When creating the VM, configure a custom metadata key named `gce-schedule-on-host` with the value `licensed-host-01` or `licensed-host-02`.

C. When creating the VM, define a node affinity label that requires the VM to be scheduled on any node within the finance department's node group.

D. When creating the VM, define a node affinity label that specifically targets the node names `licensed-host-01` or `licensed-host-02`.

O'REILLY®

Managed Instance Groups

Managed Instance Groups

Your team has developed a minor enhancement for an application running on a managed instance group in Google Compute Engine. You've created a new instance template that includes this enhancement. Since the change is non-critical, you want to ensure no currently running instances are interrupted or restarted. Instead, you want only future instances created by the managed instance group to include the update. What should you do?

A. Initiate a rolling restart to apply the update to all existing instances in sequence.

B. Trigger a rolling replacement, which will systematically delete and recreate instances with the new template.

C. Perform a rolling update using an opportunistic strategy

D. Perform a rolling update using a proactive strategy

Managed Instance Groups

Your team has developed a minor enhancement for an application running on a managed instance group in Google Compute Engine. You've created a new instance template that includes this enhancement. Since the change is non-critical, you want to ensure no currently running instances are interrupted or restarted. Instead, you want only future instances created by the managed instance group to include the update. What should you do?

A. Initiate a rolling restart to apply the update to all existing instances in sequence.

B. Trigger a rolling replacement, which will systematically delete and recreate instances with the new template.

C. Perform a rolling update using an opportunistic strategy

D. Perform a rolling update using a proactive strategy

Managed Instance Groups

Your team manages a Compute Engine managed instance group hosting a critical web application. During high traffic periods, users report slow response times and occasional timeouts. Investigation shows that the application process on each instance has reached the max CPU limit and is causing delays, but all other system processes (e.g., OS services, monitoring agents) have normal CPU and memory usage. Autoscaling has already hit its maximum number of instances configured. Downstream systems like the database are operating normally. It's important that user delays be resolved right away. What would you do?

A. Modify the autoscaling metric to trigger scaling based on memory consumption instead of CPU.

B. Disable health checks temporarily to reduce load on the instances during peak traffic.

C. Increase the CPU quota in the project to allow larger VM types to be used in the managed instance group.

D. Increase the maximum instance limit in the autoscaling policy so more instances can be provisioned under load.

Managed Instance Groups

Your team manages a Compute Engine managed instance group hosting a critical web application. During high traffic periods, users report slow response times and occasional timeouts. Investigation shows that the application process on each instance has reached the max CPU limit and is causing delays, but all other system processes (e.g., OS services, monitoring agents) have normal CPU and memory usage. Autoscaling has already hit its maximum number of instances configured. Downstream systems like the database are operating normally. It's important that user delays be resolved right away. What would you do?

A. Modify the autoscaling metric to trigger scaling based on memory consumption instead of CPU.

B. Disable health checks temporarily to reduce load on the instances during peak traffic.

C. Increase the CPU quota in the project to allow larger VM types to be used in the managed instance group.

D. Increase the maximum instance limit in the autoscaling policy so more instances can be provisioned under load.

Managed Instance Groups

Your team maintains a web service running on a single Compute Engine virtual machine. The workload is highly variable, with heavy usage during business hours and low activity otherwise. To improve responsiveness and handle traffic spikes automatically, you want to implement a scalable solution that ensures new instances are launched consistently with the current application state. What is the best approach to achieve this?

A. Create a custom image from the existing VM’s disk, create an instance template using that image, and deploy a managed instance group with autoscaling enabled based on this template.

B. Create an instance template directly from the running VM without creating a custom image, then manually scale the managed instance group during peak hours.

C. Use an instance template that references a default public image instead of the current VM’s image, then enable autoscaling on the managed instance group.

D. Take a snapshot of the VM’s disk and use it directly to launch additional instances as needed without using instance templates.

Managed Instance Groups

Your team maintains a web service running on a single Compute Engine virtual machine. The workload is highly variable, with heavy usage during business hours and low activity otherwise. To improve responsiveness and handle traffic spikes automatically, you want to implement a scalable solution that ensures new instances are launched consistently with the current application state. What is the best approach to achieve this?

A. Create a custom image from the existing VM's disk, create an instance template using that image, and deploy a managed instance group with autoscaling enabled based on this template.

B. Create an instance template directly from the running VM without creating a custom image, then manually scale the managed instance group during peak hours.

C. Use an instance template that references a default public image instead of the current VM's image, then enable autoscaling on the managed instance group.

D. Take a snapshot of the VM's disk and use it directly to launch additional instances as needed without using instance templates.

O'REILLY®

Serverless Compute Options

Servless Compute Options

Your digital marketing agency rapidly develops and tests multiple versions of landing pages for client campaigns. Developers need a way to deploy new page designs quickly. A key requirement is to empower non-technical Account Managers to instantly switch the live landing page to a new version using a simple interface. The solution must not require the agency to manage servers, clusters, or any underlying infrastructure.

Which Google Cloud product is best suited for these requirements?

A. App Engine

B. GKE On-Prem

C. Compute Engine

D. Google Kubernetes Engine

Servless Compute Options

Your digital marketing agency rapidly develops and tests multiple versions of landing pages for client campaigns. Developers need a way to deploy new page designs quickly. A key requirement is to empower non-technical Account Managers to instantly switch the live landing page to a new version using a simple interface. The solution must not require the agency to manage servers, clusters, or any underlying infrastructure.

Which Google Cloud product is best suited for these requirements?

A.App Engine

B.GKE On-Prem

C.Compute Engine

D.Google Kubernetes Engine

Serverless Compute Options

Your team is building a new media processing application. The core function of the application is to accept user-uploaded videos and apply a series of complex transformations using a specific, third-party Linux binary (e.g., a proprietary version of FFmpeg) that is not included in standard cloud platform runtimes.

You need a solution that scales automatically from zero to handle unpredictable workloads, while completely abstracting away the underlying infrastructure so your team can focus on the application logic. The deployment method must allow you to package your custom binaries along with your code.

Which Google Cloud service should you use?

A. App Engine Standard

B. Cloud Run

C. Compute Engine

D. Compute Engine with Managed Instance Groups

E. Google Kubernetes Engine (GKE)

Serverless Compute Options

Your team is building a new media processing application. The core function of the application is to accept user-uploaded videos and apply a series of complex transformations using a specific, third-party Linux binary (e.g., a proprietary version of FFmpeg) that is not included in standard cloud platform runtimes.

You need a solution that scales automatically from zero to handle unpredictable workloads, while completely abstracting away the underlying infrastructure so your team can focus on the application logic. The deployment method must allow you to package your custom binaries along with your code.

Which Google Cloud service should you use?

A. App Engine Standard

B. Cloud Run

C. Compute Engine

D. Compute Engine with Managed Instance Groups

E. Google Kubernetes Engine (GKE)

Serverless Compute Options

Your e-commerce platform runs as a service on Cloud Run. You have developed a new version of the service that includes an experimental recommendation engine. You need to perform a canary release, directing 5% of live production traffic to this new version to monitor its performance and error rates, while the other 95% of users continue to use the stable version.

You want to use the most direct and idiomatic feature of the platform to accomplish this with minimal configuration. What should you do?

A. Deploy the new version of the application as a new revision. Use the Cloud Run traffic splitting feature to direct 5% of the traffic to the new revision and leave 95% on the stable revision.

B. Create a second Cloud Run service for the experimental version. Place an external HTTPS Load Balancer in front of both the stable and experimental services and configure weighted backend routing.

C. Deploy the new version with a specific label, like version: experimental. Configure a Cloud Monitoring alert that will automatically increase traffic if the error rate is low.

D. Integrate your Cloud Run service with a service mesh like Istio. Define routing rules within the service mesh to split traffic between the stable and experimental versions of your service.

Serverless Compute Options

Your e-commerce platform runs as a service on Cloud Run. You have developed a new version of the service that includes an experimental recommendation engine. You need to perform a canary release, directing 5% of live production traffic to this new version to monitor its performance and error rates, while the other 95% of users continue to use the stable version.

You want to use the most direct and idiomatic feature of the platform to accomplish this with minimal configuration. What should you do?

A. Deploy the new revision. Use the Cloud Run traffic splitting feature to direct a fixed portion to the new version and the majority to the old version.

B. Create a second Cloud Run service for the experimental version. Place an external HTTPS Load Balancer in front of both the stable and experimental services and configure weighted backend routing.

C. Deploy the new version with a specific label, like version: experimental. Configure a Cloud Monitoring alert that will automatically increase traffic if the error rate is low.

D. Integrate your Cloud Run service with a service mesh like Istio. Define routing rules within the service mesh to split traffic between the stable and experimental versions of your service.

Cloud Functions and Cloud Run

Your company has an application that uploads images to a Cloud Storage bucket, and you need to process each image as soon as it becomes available. The processing should start automatically when a new image is uploaded, and you want to minimize costs by only paying for the compute resources when the processing happens. Additionally, your team does not want to manage any infrastructure. What should you do?

A. Deploy a managed instance group (MIG) to continuously monitor the bucket and process the images.
B. Set up a Kubernetes cluster and configure autoscaling to process images when they are uploaded.
C. Deploy your image processing code in a Cloud Function triggered by the Cloud Storage bucket.
D. Use a Cloud Run service to handle image processing, scaling based on incoming requests.

Cloud Functions and Cloud Run

Your company has an application that uploads images to a Cloud Storage bucket, and you need to process each image as soon as it becomes available. The processing should start automatically when a new image is uploaded, and you want to minimize costs by only paying for the compute resources when the processing happens. Additionally, your team does not want to manage any infrastructure. What should you do?

A. Deploy a managed instance group (MIG) to continuously monitor the bucket and process the images.
B. Set up a Kubernetes cluster and configure autoscaling to process images when they are uploaded.
C. Deploy your image processing code in a Cloud Function triggered by the Cloud Storage bucket.
D. Use a Cloud Run service to handle image processing, scaling based on incoming requests.

O'REILLY®

Google Kubernetes Engine

Google Kubernetes Engine

An online retail company runs its entire backend on a large-scale Google Kubernetes Engine (GKE) environment. The environment consists of a single cluster with a large, statically-sized node pool that runs 24/7. This cluster supports everything from the public-facing web storefront (a stateless application) and user session databases (stateful services), to end-of-day inventory processing jobs (batch workloads).

The finance department has mandated a reduction in cloud infrastructure spending, but the engineering team must not allow this to impact website uptime or responsiveness for customers. What is the most effective strategy to meet these requirements?

A. Implement a two-tiered autoscaling strategy: Use the Horizontal Pod Autoscaler (HPA) and enable the GKE Cluster Autoscaler to dynamically resize the node pool as needed.
B. Re-architect the environment by provisioning a second, smaller GKE cluster exclusively for the inventory processing jobs, and manually re-allocate some nodes from the main cluster to the new one.
C. Focus on workload stability by defining strict CPU and memory requests and limits for every deployment in the cluster, ensuring no single pod can consume excessive resources.
D. Achieve maximum cost savings by converting the entire node pool to use Spot VMs, forcing all workloads including the stateful databases and web storefront to run on nodes that can be terminated at any time.

Google Kubernetes Engine

An online retail company runs its entire backend on a large-scale Google Kubernetes Engine (GKE) environment. The environment consists of a single cluster with a large, statically-sized node pool that runs 24/7. This cluster supports everything from the public-facing web storefront (a stateless application) and user session databases (stateful services), to end-of-day inventory processing jobs (batch workloads).

The finance department has mandated a reduction in cloud infrastructure spending, but the engineering team must not allow this to impact website uptime or responsiveness for customers. What is the most effective strategy to meet these requirements?

A. **Implement a two-tiered autoscaling strategy: Use the Horizontal Pod Autoscaler (HPA) and enable the GKE Cluster Autoscaler to dynamically resize the node pool as needed**.
B. Re-architect the environment by provisioning a second, smaller GKE cluster exclusively for the inventory processing jobs, and manually re-allocate some nodes from the main cluster to the new one.
C. Focus on workload stability by defining strict CPU and memory requests and limits for every deployment in the cluster, ensuring no single pod can consume excessive resources.
D. Achieve maximum cost savings by converting the entire node pool to use Spot VMs, forcing all workloads including the stateful databases and web storefront to run on nodes that can be terminated at any time.

Google Kubernetes Engine

You are designing a backend order-processing system on Google Kubernetes Engine. The system consists of multiple internal components, including an order-intake service and a payment-processing service. The order-intake service needs to reliably communicate with the payment-processing service.

1. The payment-processing service is resilient and can be scaled up or down by simply changing a replica count.
2. The order-intake service can use a single, consistent address to reach the payment service, and Kubernetes will automatically load-balance requests across all available payment service replicas.
3. Communication between these services remains private and internal to the cluster.

A. For the payment-processing service, create a **Deployment**. To make it accessible, create an **Ingress** resource to expose it to the public internet, and have the order-intake service communicate via the Ingress's public IP address.
B. For the payment-processing service, manually create several individual **Pods**. Then, create a **Service** that uses a label selector to group these pods and provide a single DNS name for communication.
C. For the payment-processing service, create a **StatefulSet** to manage its pods. Then, create a **Gateway** resource to manage traffic routing from the order-intake service.
D. For the payment-processing service, create a **Deployment**. Then, create a **Service** (of type ClusterIP) that targets the Deployment's pods. The order-intake service will use the auto-generated DNS name of this Service.

Google Kubernetes Engine

You are designing a backend order-processing system on Google Kubernetes Engine. The system consists of multiple internal components, including an order-intake service and a payment-processing service. The order-intake service needs to reliably communicate with the payment-processing service.

1. The payment-processing service is resilient and can be scaled up or down by simply changing a replica count.
2. The order-intake service can use a single, consistent address to reach the payment service, and Kubernetes will automatically load-balance requests across all available payment service replicas.
3. Communication between these services remains private and internal to the cluster.

A. For the payment-processing service, create a **Deployment**. To make it accessible, create an **Ingress** resource to expose it to the public internet, and have the order-intake service communicate via the Ingress's public IP address.

B. For the payment-processing service, manually create several individual **Pods**. Then, create a **Service** that uses a label selector to group these pods and provide a single DNS name for communication.

C. For the payment-processing service, create a **StatefulSet** to manage its pods. Then, create a **Gateway** resource to manage traffic routing from the order-intake service.

D. For the payment-processing service, create a Deployment. Then, create a Service (of type ClusterIP) that targets the Deployment's pods. The order-intake service will use the auto-generated DNS name of this Service.

Google Kubernetes Engine

A financial services company runs its critical payment application on a production Google Kubernetes Engine (GKE) cluster. For strict compliance, the company must enforce a policy that only allows container images to be deployed if they have passed two specific quality gates:

1. A successful vulnerability scan from an automated security tool.
2. A manual sign-off from the Quality Assurance (QA) team.

How do you prevent the deployment of images who have not passed these checks?

A. In the application's Deployment manifest, add a postStart lifecycle hook that calls an external script to verify the image's approval status and stops the container if it fails.

B. Use Cloud Source Repositories to host the container images and configure IAM permissions so that only the QA team lead has rights to deploy to the production cluster.

C. Use Binary Authorization to define a policy for the production cluster that attestations for each quality gate before an image can be deployed. Integrate the creation of these attestations into your CI/CD process.

D. Write and deploy a custom validating admission webhook. The webhook's logic will query an external database of approved images to either admit or deny each pod creation request.

Google Kubernetes Engine

A financial services company runs its critical payment application on a production Google Kubernetes Engine (GKE) cluster. For strict compliance, the company must enforce a policy that only allows container images to be deployed if they have passed two specific quality gates:

1. A successful vulnerability scan from an automated security tool.
2. A manual sign-off from the Quality Assurance (QA) team.

How do you prevent the deployment of images who have not passed these checks?

A. In the application's Deployment manifest, add a postStart lifecycle hook that calls an external script to verify the image's approval status and stops the container if it fails.

B. Use Cloud Source Repositories to host the container images and configure IAM permissions so that only the QA team lead has rights to deploy to the production cluster.

C. Use Binary Authorization to define a policy for the production cluster that attestations for each quality gate before an image can be deployed. Integrate the creation of these attestations into your CI/CD process.

D. Write and deploy a custom validating admission webhook. The webhook's logic will query an external database of approved images to either admit or deny each pod creation request.

Google Kubernetes Engine

A gaming company hosts the backend for its popular real-time multiplayer game on a GKE cluster in europe-west1. Following a successful launch in North America, players there are experiencing high latency (lag), which negatively impacts gameplay.

You need to create a low-latency experience for players in both regions by directing them to the nearest healthy cluster. The game backend is a stateless application.

What is the best way to architect this multi-region deployment on Google Cloud?

A. Create a new GKE cluster in a North American region. In each cluster, expose the game server via a Service of type LoadBalancer. In Cloud DNS, create A records with the same hostname for both public IPs and rely on DNS-based routing.

B. Create a new GKE cluster in a North American region. Register both clusters to a GKE Fleet and configure Multi-Cluster Ingress to deploy a single global load balancer that intelligently routes player traffic to the nearest available regional cluster.

C. Keep the single GKE cluster in europe-west1 but place a Global External HTTP/S Load Balancer in front of it and enable Cloud CDN to cache static game assets closer to North American players.

D. Keep the single GKE cluster in europe-west1 and enable Horizontal Pod Autoscaling to ensure there are enough game server pods to handle the increased player load from both continents.

Google Kubernetes Engine

A gaming company hosts the backend for its popular real-time multiplayer game on a GKE cluster in europe-west1. Following a successful launch in North America, players there are experiencing high latency (lag), which negatively impacts gameplay.

You need to create a low-latency experience for players in both regions by directing them to the nearest healthy cluster. The game backend is a stateless application.

What is the best way to architect this multi-region deployment on Google Cloud?

A. Create a new GKE cluster in a North American region. In each cluster, expose the game server via a Service of type LoadBalancer. In Cloud DNS, create A records with the same hostname for both public IPs and rely on DNS-based routing.

B. Create a new GKE cluster in a North American region. Register both clusters to a GKE Fleet and configure Multi-Cluster Ingress to deploy a single global load balancer that intelligently routes player traffic to the nearest available regional cluster.

C. Keep the single GKE cluster in europe-west1 but place a Global External HTTP/S Load Balancer in front of it and enable Cloud CDN to cache static game assets closer to North American players.

D. Keep the single GKE cluster in europe-west1 and enable Horizontal Pod Autoscaling to ensure there are enough game server pods to handle the increased player load from both continents.

O'REILLY®

Load Balancing

Load Balancing

Your team needs to deploy a legacy financial application on Google Cloud. The application communicates over TCP and relies on direct access to a local filesystem to maintain data integrity. It cannot be scaled horizontally because it does not have synchronization and accessing the file system concurrently causes problems that cannot be resolved easily. Although brief downtime is acceptable during failover, the application must remain highly available to support continuous business operations. How should you architect this deployment?

A. Deploy the application on a managed instance group across multiple zones, use Cloud Filestore for shared storage, and place an HTTP(S) load balancer in front to distribute traffic.

B. Deploy the application on a managed instance group across multiple zones, use Cloud Filestore for shared storage, and use a network load balancer to balance traffic.

C. Create an unmanaged instance group with one active and one standby VM in different zones, attach a regional persistent disk for storage, and place an HTTP(S) load balancer in front to route requests.

D. Create an unmanaged instance group with an active VM and a standby VM in separate zones, use a regional persistent disk for data storage, and place a network load balancer in front to route client connections.

Load Balancing

Your team needs to deploy a legacy financial application on Google Cloud. The application communicates over TCP and relies on direct access to a local filesystem to maintain data integrity. It cannot be scaled horizontally because it does not have synchronization and accessing the file system concurrently causes problems that cannot be resolved easily. Although brief downtime is acceptable during failover, the application must remain highly available to support continuous business operations. How should you architect this deployment?

A. Deploy the application on a managed instance group across multiple zones, use Cloud Filestore for shared storage, and place an HTTP(S) load balancer in front to distribute traffic.

B. Deploy the application on a managed instance group across multiple zones, use Cloud Filestore for shared storage, and use a network load balancer to balance traffic.

C. Create an unmanaged instance group with one active and one standby VM in different zones, attach a regional persistent disk for storage, and place an HTTP(S) load balancer in front to route requests.

D. Create an unmanaged instance group with an active VM and a standby VM in separate zones, use a regional persistent disk for data storage, and place a network load balancer in front to route client connections.

Load Balancing

A media company is building a global video streaming platform on Google Cloud using dozens of microservices. They require a CI/CD pipeline for frequent, independent updates using immutable artifacts. The architecture must provide low-latency access to users in both North America and Asia via a single global entry point, while keeping core backend services private from the internet. Which set of Google Cloud services is most suitable for this architecture?

A. Artifact Registry to store container images, Google Kubernetes Engine (GKE) clusters in an Asian and North American region, and a Global External HTTP/S Load Balancer.

B. App Engine to host the services in multiple regions, Cloud Spanner for a global database, and a regional Internal TCP/UDP Load Balancer.

C. Cloud Storage to store application binaries, Managed Instance Groups (MIGs) in each region, and a Regional External Network Load Balancer for each region.

D. Cloud Functions for the microservice logic and a Global External HTTP/S Load Balancer configured with Serverless Network Endpoint Groups (NEGs).

Load Balancing

A media company is building a global video streaming platform on Google Cloud using dozens of microservices. They require a CI/CD pipeline for frequent, independent updates using immutable artifacts. The architecture must provide low-latency access to users in both North America and Asia via a single global entry point, while keeping core backend services private from the internet. Which set of Google Cloud services is most suitable for this architecture?

A. **Artifact Registry to store container images, Google Kubernetes Engine (GKE) clusters in an Asian and North American region, and a Global External HTTP/S Load Balancer.**

B. App Engine to host the services in multiple regions, Cloud Spanner for a global database, and a regional Internal TCP/UDP Load Balancer.

C. Cloud Storage to store application binaries, Managed Instance Groups (MIGs) in each region, and a Regional External Network Load Balancer for each region.

D. Cloud Functions for the microservice logic and a Global External HTTP/S Load Balancer configured with Serverless Network Endpoint Groups (NEGs).

Load Balancing

An international e-commerce company has deployed its containerized shopping cart API on Cloud Run in two regions: europe-west1 and australia-southeast1. They need to provide a single global endpoint (api.shopping.com) for their frontend applications that automatically routes customers to the closest healthy region, ensuring a fast and resilient shopping experience.

What is the recommended Google Cloud native approach to achieve this?

A. Create a **Regional External HTTP/S Load Balancer** in both regions. Use Cloud DNS to create weighted A records to distribute traffic between the two load balancer IPs.

B. For each regional Cloud Run service, create a **Serverless Network Endpoint Group (NEG)**. Create a single **Global External HTTP/S Load Balancer** and configure its backend service to use these two Serverless NEGs.

C. Use **API Gateway** in front of each Cloud Run service. Configure the two API Gateway instances under a single custom domain in Cloud DNS to handle routing.

D. Create a **Global External HTTP/S Load Balancer**. Manually add the default .run.app URLs of the two Cloud Run services as **Internet Network Endpoint Group** backends to the load balancer.

Load Balancing

An international e-commerce company has deployed its containerized shopping cart API on Cloud Run in two regions: europe-west1 and australia-southeast1. They need to provide a single global endpoint (api.shopping.com) for their frontend applications that automatically routes customers to the closest healthy region, ensuring a fast and resilient shopping experience.

What is the recommended Google Cloud native approach to achieve this?

A. Create a **Regional External HTTP/S Load Balancer** in both regions. Use Cloud DNS to create weighted A records to distribute traffic between the two load balancer IPs.

B. For each regional Cloud Run service, create a **Serverless Network Endpoint Group (NEG)**. Create a single **Global External HTTP/S Load Balancer** and configure its backend service to use these two Serverless NEGs.

C. Use **API Gateway** in front of each Cloud Run service. Configure the two API Gateway instances under a single custom domain in Cloud DNS to handle routing.

D. Create a **Global External HTTP/S Load Balancer**. Manually add the default .run.app URLs of the two Cloud Run services as **Internet Network Endpoint Group** backends to the load balancer.